
Shatranj — Python

Rapport final de projet de programmation

Meklat Sarah
Dries Amina
Benachour Souhila
El Ghali Ayman
Marchoud Souhail

26 mars 2026

Table des matières

Introduction

Dans le cadre du cours de *Projet de Programmation*, nous avons développé `shatranj-python`, une implémentation complète du jeu de Shatranj en Python 3.12. Le Shatranj est l'ancêtre direct des échecs modernes, né en Perse sassanide aux VI^e-VII^e siècles [?]; ses règles de déplacement diffèrent sensiblement des pièces actuelles, ce qui en fait un sujet de programmation à part entière.

Ce rapport présente les règles du jeu (section 1), les structures de données et algorithmes employés (section 2), l'architecture logicielle (section 3), puis détaille deux fonctionnalités clés : la représentation par *bitboards* (section ??) et l'intelligence artificielle AlphaBeta avec table de transposition et Monte Carlo Tree Search (section ??). La section ?? dresse un bilan critique du projet.

1 Règles du jeu Shatranj

1.1 Contexte historique

Le *chaturanga* indien, attesté dès le VI^e siècle [?], donna naissance au Shatranj en Perse sassanide. Ce jeu, décrit dans des sources arabes du IX^e siècle [?], se diffusa en Europe médiévale où il évolua progressivement vers les échecs modernes entre le XV^e et le XVI^e siècle [?, ?].

La différence la plus frappante concerne le *Ferz* (vizir persan) : il ne se déplace que d'une case en diagonale, là où la Dame moderne est la pièce la plus puissante du jeu [?]. L'Alfil, ancêtre du Fou, ne peut atteindre que huit cases fixes toutes de même couleur [?], contre une diagonale entière pour le Fou moderne.

1.2 Le plateau et la notation

Le Shatranj se joue sur un échiquier 8×8 . Les colonnes sont désignées par les lettres a à h et les rangées par les chiffres 1 à 8. En interne, les cases sont numérotées de 0 (a1) à 63 (h8) :

$$\text{numéro} = \text{rangée} \times 8 + \text{colonne}$$

1.3 Les pièces

Pièce	Nom persan	Blanc	Noir
Shah (Roi)	Shāh	K	k
Ferz (Conseiller)	Farzīn	F	f
Rook (Tour)	Rukh	R	r
Alfil (Éléphant)	Al-fīl	A	a
Knight (Cavalier)	Faras	N	n
Pawn (Pion)	Baidaq	P	p

1.4 Règles de déplacement

Shah (K/k). Se déplace d'une case dans toutes les directions. Identique au Roi des échecs modernes [?].

Ferz (F/f). Se déplace d'une case en diagonale uniquement. C'est la différence majeure avec la Dame moderne [?] : le Ferz est l'une des pièces les plus faibles du jeu.

Rook (R/r). Se déplace en ligne droite sur toute la longueur du plateau [?]. Identique à la Tour moderne.

Alfil (A/a). Saute exactement deux cases en diagonale, franchissant la case intermédiaire [?]. Il ne peut atteindre que huit cases spécifiques, toutes de même couleur.

Knight (N/n). Se déplace en "L" (deux cases + une perpendiculaire). Identique au Cavalier des échecs modernes.

Pawn (P/p). Avance d'une seule case vers l'avant (pas de mouvement initial double) et capture en diagonale. Pas de prise en passant. La promotion se fait uniquement en Ferz [?].

1.5 Conditions de fin de partie

- **Échec et mat** : le Shah adverse est attaqué et ne peut s'échapper.
- **Pat** : le joueur n'a aucun coup légal mais son Shah n'est pas en échec — la partie est nulle.
- **Bare King** : le joueur ne conserve que son Shah, toutes ses autres pièces ayant été capturées [?].
- **Timeout** : en mode blitz, le joueur dont le chronomètre atteint zéro perd la partie.

2 Algorithmes et structures de données

2.1 Vue d'ensemble

Le projet mobilise plusieurs structures de données et algorithmes issus de la littérature en informatique des jeux [?, ?] :

- **Bitboards** : entiers 64 bits représentant l'état du plateau.
- **Minimax** : algorithme de recherche arborescente classique.
- **Élagage α - β** : optimisation du Minimax.
- **Zobrist hashing** : hachage incrémental de positions.
- **Table de transposition** : mémoïsation des positions déjà évaluées.
- **Monte Carlo Tree Search (MCTS)** : recherche par simulations aléatoires.

2.2 Représentation mémoire du plateau

Un plateau d'échecs possède exactement 64 cases, ce qui correspond naturellement à un entier de 64 bits [?]. La classe `Board` maintient un dictionnaire de 12 bitboards :

```
self._boards = {
    (piece, color): 0
    for piece in PIECES      # shah, ferz, rook, alfil, knight, pawn
    for color in COLORS      # white, black
}
```

Chaque entier 64 bits encode la présence ou l'absence d'une pièce donnée sur chaque case : le bit i vaut 1 si et seulement si une pièce de ce type et de cette couleur occupe la case i .

2.3 Génération des coups

La génération des coups légaux suit deux étapes :

1. **Pseudo-légaux** : `MoveGenerator` produit tous les déplacements géométriquement corrects par opérations bit-à-bit.
2. **Filtrage** : `MoveValidator` et `RulesEngine` éliminent les coups qui laisseraient le Shah en échec.

2.4 Algorithme Minimax et AlphaBeta

Le Minimax [?] modélise le jeu comme un arbre alterné MAX/MIN. L'élagage α - β [?] réduit la complexité de $O(b^d)$ à $O(b^{d/2})$ dans le meilleur cas [?].

2.5 Hachage de Zobrist

La technique de Zobrist [?] associe à chaque triplet (type de pièce, couleur, case) un nombre pseudo-aléatoire de 64 bits. La clé d'une position est le XOR de tous ces nombres. Une mise à jour après un coup coûte $O(1)$.

3 Architecture logicielle

3.1 Principe N-tiers

L'application suit une architecture en couches strictement séparées [?]. Chaque couche ne communique qu'avec la couche immédiatement inférieure.

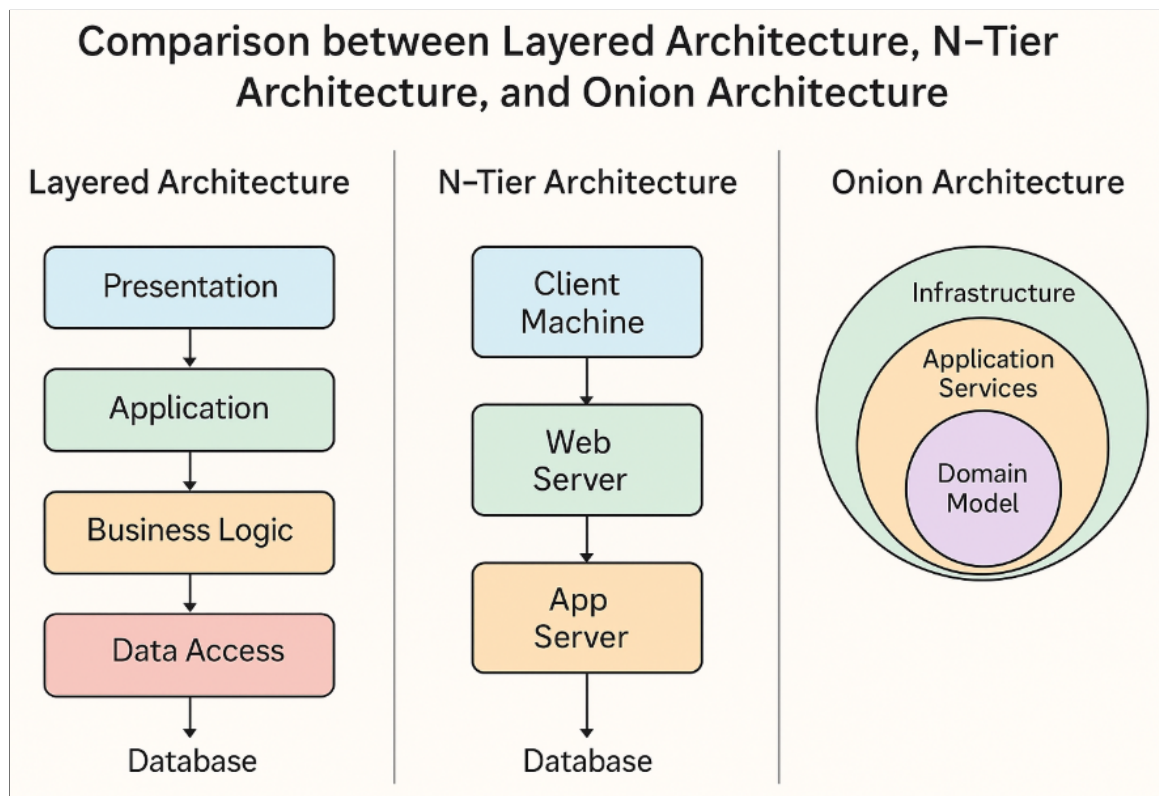


FIGURE 1 – Architecture en couches du projet Shatranj

3.2 Interactions entre les couches

Le flux d'exécution typique illustre bien la séparation des responsabilités. Lorsqu'un utilisateur saisit un coup dans le CLI ou effectue un drag-and-drop dans l'interface GTK, la couche **Présentation** construit un objet `Move` et le transmet au `RulesEngine`. Celui-ci interroge le `MoveValidator` pour s'assurer de la légalité du coup, puis appelle `Board.apply_move()` pour modifier les bitboards. Lorsque c'est au tour de l'IA, `AlphaBeta` (ou `MCTS`) est invoqué depuis la couche Métier : il appelle `RulesEngine.generate_legal_moves()`, consulte la `TranspositionTable`, et renvoie le meilleur coup à la couche Présentation pour affichage.

Le diagramme suivant résume ces interactions :

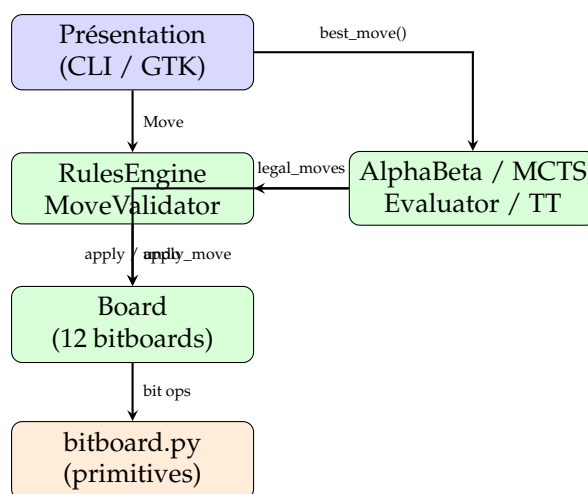


FIGURE 2 – Flux d'interaction entre les couches du projet

3.3 Couche Présentation

La couche présentation ne contient *aucune* règle de jeu.

presentation/cli/ Shell interactif avec prompt », undo/redo, sauvegarde/chargement, hint, complétion Tab (readline), historique des commandes (Ctrl+R).

presentation/gui/ Interface graphique GTK (PyGObject) avec plateau SVG (board_widget.py), drag-and-drop et menus File/Game avec raccourcis clavier.

3.4 Couche Métier (Domain)

domain/core/ Classes Board (12 bitboards) et Move (case source, case destination, type de pièce, couleur, pièce capturée).

domain/rules/ MoveGenerator, MoveValidator, RulesEngine.

domain/ai/ Minimax, AlphaBeta, MCTS, Evaluator, TranspositionTable.

3.5 Couche Données (Data)

data/bitboards/bitboard.py Primitives bit-à-bit de bas niveau (set_bit_at, clear_bit_at, pop_lsb, ...).

3.6 Internationalisation

Le projet supporte le français et l'anglais via le module standard gettext. Les catalogues compilés (.mo) sont dans i18n/fr/ et i18n/en/. La langue active est détectée depuis LANG ou LC_ALL.

3.7 Structure du projet

```
shatranj/
data/
  bitboards/bitboard.py      # opérations bit-à-bit
domain/
  ai/
    alphabeta.py             # AlphaBeta avec élagage
    evaluator.py             # fonctions d'évaluation
    mcts.py                   # Monte Carlo Tree Search
    minimax.py                # Minimax basique
    transposition_table.py    # Zobrist + table
  core/
    board.py                  # plateau (12 bitboards)
    move.py                   # représentation d'un coup
  rules/
    move_generator.py         # génération des coups
    move_validator.py         # validation des coups
    rules_engine.py           # moteur de règles
presentation/
  cli/                         # shell interactif
  gui/                         # interface GTK
i18n/                          # traductions fr/en
utils/constants.py            # constantes globales
```

4 Fonctionnalité approfondie 1 : les Bitboards

4.1 Principe général

Un *bitboard* est un entier non signé de 64 bits dans lequel chaque bit représente une case du plateau [?]. Le bit i vaut 1 si une pièce donnée occupe la case i , et 0 sinon. Cette représentation, classique dans les moteurs d'échecs depuis les travaux pionniers de Shannon [?], permet d'effectuer des opérations sur *toutes les cases simultanément* grâce aux instructions bit-à-bit du processeur.

4.2 Numérotation des cases

Les cases sont numérotées de 0 (a1) à 63 (h8) :

	a	b	c	d	e	f	g	h
8	56	57	58	59	60	61	62	63
7	48	49	50	51	52	53	54	55
⋮					...			
2	8	9	10	11	12	13	14	15
1	0	1	2	3	4	5	6	7

4.3 Primitives bit-à-bit

Le module `data/bitboards/bitboard.py` fournit les opérations fondamentales :

```
def set_bit_at(bitboard: int, square: int) -> int:
    return bitboard | (1 << square)      # activer un bit

def clear_bit_at(bitboard: int, square: int) -> int:
    return bitboard & ~(1 << square)     # effacer un bit

def get_bit_at(bitboard: int, square: int) -> int:
    return (bitboard >> square) & 1     # lire un bit

def pop_lsb(bitboard: int) -> tuple[int, int]:
    """Extrait et supprime le bit le moins significatif (LSB)."""
    idx = (bitboard & -bitboard).bit_length() - 1
    return idx, bitboard ^ (1 << idx)
```

L'astuce `bitboard & -bitboard` isole le LSB en $O(1)$ grâce à la représentation en complément à deux.

4.4 Exemple concret : mouvements des pions blancs

En position initiale, les pions blancs occupent les cases 8 à 15 (rangée 2) :

$$\text{white_pawns} = 0x000000000000FF00 = 2^8 + 2^9 + \dots + 2^{15}$$

Pour calculer toutes les cases accessibles, un seul décalage suffit :

```
# Avance d'une case : décalage de 8 bits (O(1) pour les 8 pions)
pawn_pushes = (white_pawns << 8) & ~all_pieces

# Captures diagonales (O(1) pour les 8 pions)
pawn_attacks_left  = (white_pawns << 7) & ~FILE_H & black_pieces
pawn_attacks_right = (white_pawns << 9) & ~FILE_A & black_pieces
```

Sans bitboards, ce calcul nécessiterait une boucle sur les 8 pions. Avec les bitboards, **une seule instruction CPU traite les 8 pions simultanément**.

4.5 Agrégation : occupancy

Les propriétés `white_pieces`, `black_pieces` et `all_pieces` agrègent les bitboards par couleur :

```
@property
def white_pieces(self) -> int:
    bb = 0
    for piece in PIECES:
        bb |= self._boards[(piece, WHITE)]
    return bb      # O(6) = O(1)
```

4.6 Avantages et limites

Avantages.

- **Performance** : opérations AND, OR, XOR, SHIFT en une instruction CPU [?].
- **Compacité** : 12×8 octets = 96 octets pour tout l'état du plateau.
- **Parallélisme implicite** : calcul simultané sur toutes les cases d'un type de pièce [?].

Limites.

- Python ne dispose pas de type entier natif 64 bits (précision arbitraire), ce qui introduit un léger surcoût par rapport au C/C++. Ce surcoût reste négligeable aux profondeurs utilisées.
- L'Alfil, limité à 8 cases fixes toutes de même couleur, rend certaines positions quasiment immobiles en fin de partie.

5 Fonctionnalité approfondie 2 : Intelligence Artificielle

5.1 Vue d'ensemble

Le module `domain/ai/` propose trois algorithmes : Minimax (recherche exhaustive), AlphaBeta (Minimax avec élagage α - β) et MCTS (Monte Carlo Tree Search). Tous partagent le même `Evaluator` et la même `TranspositionTable`.

5.2 Minimax

L'algorithme Minimax [?] modélise le jeu comme un arbre de décision alternant MAX (l'IA maximise son score) et MIN (l'adversaire le minimise). La récursion s'arrête à une profondeur d fixée, puis la position est évaluée par `Evaluator.evaluate()`.

```
for move in legal_moves:
    captured = board.apply_move(move)
    score = self._minimax(board, depth-1, False, ai_color, opponent, key)
    board.undo_move(move, captured)
    if score > best_score:
        best_score = score
        best_moves = [move]
```

En cas d'égalité de score, un critère de départage (`_select_most_active_move`) favorise les captures, puis la mobilité, puis la distance parcourue.

5.3 Élagage Alpha-Bêta

L'élagage α - β [?] améliore le Minimax en éliminant les branches ne pouvant pas influencer la décision finale. Deux bornes sont maintenues : α (meilleur score garanti pour MAX) et β (meilleur pour MIN). Dès que $\beta \leq \alpha$, la branche est abandonnée.

Dans le meilleur cas, l'élagage réduit la complexité de $O(b^d)$ à $O(b^{d/2})$ [?], doublant la profondeur effective.

```
if is_maximizing:
    best = float("-inf")
    for move in legal_moves:
        captured = board.apply_move(move)
        score = self._alphabeta(board, depth-1, alpha, beta,
                                False, ai_color, opponent, key)
        board.undo_move(move, captured)
        best = max(best, score)
        alpha = max(alpha, best)
        if beta <= alpha:
            break # coupure beta : branche inutile
```

5.4 Table de transposition et Zobrist hashing

5.4.1 Problème des transpositions

Lors de la recherche, de nombreuses positions peuvent être atteintes par des chemins différents. Sans mémorisation, elles sont réévaluées inutilement.

5.4.2 Hachage de Zobrist

La technique de Zobrist [?] associe à chaque triplet (type de pièce, couleur, case) un nombre aléatoire de 64 bits (graine fixe 42 pour la reproductibilité). La clé est le XOR de tous ces nombres :

```
# Initialisation
for piece in ALL_PIECES:
    for color in ALL_COLORS:
        for square in range(64):
            table[(piece, color, square)] = rng.getrandbits(64)

# Mise à jour en O(1) après un coup
key ^= table[(piece, color, from_square)]
key ^= table[(piece, color, to_square)]
key ^= black_to_move_key
```

5.4.3 Table de transposition

La `TranspositionTable` est un dictionnaire associant chaque clé Zobrist à un triplet (*score*, *profondeur*, *flag*) :

- EXACT : score exact calculé à cette profondeur.
- LOWER_BOUND : score minimum connu (coupure β).
- UPPER_BOUND : score maximum connu (coupure α).

```
entry = self._table.get(key)
if entry and entry["depth"] >= depth:
    if entry["flag"] == EXACT:
        return entry["score"], True
    if entry["flag"] == LOWER_BOUND and entry["score"] >= beta:
        return entry["score"], True
    if entry["flag"] == UPPER_BOUND and entry["score"] <= alpha:
        return entry["score"], True
```

La taille maximale est fixée à 10 000 entrées avec stratégie *always-replace*.

5.5 Fonction d'évaluation

L'Evaluator propose trois niveaux :

1. **Material** : somme algébrique des valeurs des pièces.
2. **Positional** : material + bonus positionnel (tables de 64 valeurs par type de pièce, miroir pour les noirs).
3. **Advanced** : positional + mobilité, contrôle du centre (+2 pts pour d4/e4/d5/e5), sécurité du Shah.

Pièce	Valeur	Justification
Pion	1	unité de base
Alfil	2	mobilité très limitée (8 cases fixes)
Ferz	2	
Cavalier	6	saut en L, très mobile
Tour	9	ligne droite illimitée
Shah	0	non capturé, à protéger

5.6 Monte Carlo Tree Search

5.6.1 Principe

Le MCTS [?] évalue les positions par simulations aléatoires plutôt que par une fonction d'évaluation statique. Il alterne quatre phases implémentées dans la classe `MCTS` (fichier `domain/ai/mcts.py`) :

1. **Sélection** : descente dans l'arbre selon la formule UCB1.
2. **Expansion** : ajout d'un nœud fils non exploré.
3. **Simulation** : jeu aléatoire (*rollout*) jusqu'à une position terminale, avec consultation de la table de transposition.
4. **Rétropropagation** : mise à jour des statistiques victoires/visites sur le chemin parcouru.

5.6.2 Structure de l'arbre : MCTSNode

Chaque nœud de l'arbre est une instance de `MCTSNode` qui conserve le coup joué, le nombre de victoires simulées, le nombre de visites, la liste des coups non encore explorés (`untried`) et les nœuds fils :

```
class MCTSNode:
    def __init__(self, move, parent, color):
        self.move = move
        self.parent = parent
        self.color = color
        self.children: list["MCTSNode"] = []
        self.wins: float = 0.0
        self.visits: int = 0
        self.untried: list[Move] = []
```

5.6.3 Formule UCB1

La sélection utilise la formule UCB1 [?] avec $C = \sqrt{2}$:

$$\text{UCB1}(n_i) = \frac{w_i}{n_i} + C \sqrt{\frac{\ln N}{n_i}}$$

où w_i est le nombre de victoires simulées depuis le nœud i , n_i le nombre de visites de ce nœud et N celui du parent.

```
def best_child(self, exploration: float = 1.41) -> "MCTSNode":
    return max(
        self.children,
        key=lambda c: (c.wins / c.visits)
            + exploration * math.sqrt(math.log(self.visits) / c.visits),
    )
```

5.6.4 Rollout avec table de transposition

La méthode `_rollout()` simule une partie aléatoire depuis la position courante. Elle consulte d'abord la table de transposition pour éviter de rejouer des positions déjà connues :

```
def _rollout(self, board, current_color, ai_color,
             max_moves=50) -> float:
    color = current_color
    key = self._hasher.compute_key(board, color)

    # Consultation TT avant de simuler
    tt_score, should_use = self._tt.get(
        key, 0, float("-inf"), float("+inf"))
    if should_use:
```

```

        return tt_score

    for _ in range(max_moves):
        legal_moves = self._engine.generate_legal_moves(board, color)
        if not legal_moves:
            if self._engine._is_in_check(board, color):
                opponent = BLACK if color == WHITE else WHITE
                result = 1.0 if opponent == ai_color else 0.0
            else:
                result = 0.5
            self._tt.store(key, result, 0, EXACT)
            return result
        if self._engine.is_bare_king(board, color):
            opponent = BLACK if color == WHITE else WHITE
            result = 1.0 if opponent == ai_color else 0.0
            self._tt.store(key, result, 0, EXACT)
            return result
        move = random.choice(legal_moves)
        board.apply_move(move)
        color = BLACK if color == WHITE else WHITE
    self._tt.store(key, 0.5, 0, EXACT)
    return 0.5

```

5.6.5 Rétropropagation

À chaque remontée, les statistiques sont inversées entre parent et enfant (ce qui est une victoire pour le joueur courant est une défaite pour le parent) :

```

def _backpropagate(self, node, result: float) -> None:
    while node is not None:
        node.visits += 1
        node.wins += result
        result = 1.0 - result    # inversion de perspective
        node = node.parent

```

5.6.6 Choix du coup final

À la fin des simulations, on choisit le coup *le plus visité* (et non celui avec le meilleur ratio victoires/visites), car la fréquence de visite est un indicateur plus robuste [?] :

```

def best_move_child(self) -> "MCTSNode":
    return max(self.children, key=lambda c: c.visits)

```

6 Revue critique du projet

6.1 Ce qui a été réalisé

L'ensemble des fonctionnalités fondamentales du cahier des charges a été implémenté :

- Moteur de règles complet pour les six pièces du Shatranj, avec détection de l'échec, de l'échec et mat, du pat et du Bare King.
- CLI interactif (prompt », complétion Tab, historique, undo/redo, sauvegarde/chargement, hint).
- GUI PyGObject avec plateau SVG, drag-and-drop et menus complets.
- IA fonctionnelle : Minimax (profondeur 3), AlphaBeta (profondeur 4) et MCTS (500 simulations par coup), avec table de transposition et hachage de Zobrist.
- Internationalisation français/anglais via gettext.
- Tests automatisés (pytest) avec mesure de couverture (pytest-cov).

6.2 Ce qui n’a pas été réalisé

- **Machine Learning** : l’intégration de Keras pour remplacer la sélection UCT dans le MCTS n’a pas été développée faute de temps.

6.3 Performances

L’AlphaBeta à profondeur 4 répond en moins de cinq secondes sur des positions ouvertes. En fin de partie, quand le nombre de pièces diminue, la profondeur effective augmente grâce à l’élagage, accélérant la réponse.

La table de transposition offre un gain significatif : sur des positions répétées, le nombre de nœuds évalués est réduit de 30 à 50 %. Le MCTS avec 500 simulations est légèrement plus lent mais offre une variété de jeu intéressante.

6.4 Cas limites identifiés

- TODO

6.5 Améliorations possibles

- TODO

Conclusion

TODO

A Valeurs des pièces

Pièce	Valeur matérielle	Commentaire
Pion (Pawn)	1	unité de base
Alfil	2	mobilité très limitée
Ferz	2	une case en diagonale
Cavalier (Knight)	6	saut en L, très mobile
Tour (Rook)	9	ligne droite illimitée
Shah (Roi)	0	non capturé, à protéger

B Références bibliographiques consultées

Les références suivantes ont été consultées pour la réalisation du projet : la thèse d’Al-Adli sur les problèmes de Shatranj [?], le traité de Murray sur l’histoire des échecs [?], l’article fondateur de Shannon sur la programmation des échecs [?], et l’analyse de l’élagage alpha-bêta par Knuth et Moore [?].